

注意力机制

数据科学与大数据技术专业

第 9 讲

1 导读

除本导读外, 本讲共有 6 个主体节. 它们从注意力机制的动机讲起, 逐步建立人工神经网络中的注意力计算, 再说明注意力在文本、翻译、视觉和阅读理解中的应用, 最后进入自注意力、Transformer 以及生成式语言模型的内部结构.

1. **注意力机制**: 说明为什么除了加深、加宽网络外, 还需要一种能动态选择关键信息的机制.
2. **人工神经网络的注意力机制**: 说明注意力分布、打分函数、键值对注意力、多头注意力和指针网络的基本形式.
3. **注意力机制的应用**: 说明注意力如何用于文本分类、层次文档建模、机器翻译、图像描述和阅读理解.
4. **自注意力模型**: 说明如何用注意力在同一序列内部建立任意位置之间的依赖关系, 以及其矩阵化实现.
5. **Transformer 简介**: 说明 Transformer 块为什么需要位置编码、残差连接、层归一化和逐位前馈网络, 并比较 CNN、RNN 与 Transformer.
6. **生成式模型的细节**: 说明现代生成式语言模型中分词器、Transformer 块和语言建模头各自承担什么作用.

如果细节都忘了: 最应该记住的是, 注意力机制不是简单地“看哪里”, 而是通过可学习的相关性打分为不同输入分配权重, 再把相关信息加权汇聚成当前需要的表示. 自注意力把这个过程推广到序列内部所有位置之间, Transformer 则在此基础上构成了现代大语言模型的核心计算模块.

问题思考:

- 为什么同样是增加模型能力, 注意力机制与单纯增加网络层数或参数量有本质不同?

2 注意力机制

本节导读:

- **本节目的**: 建立注意力机制的直观动机, 说明它为什么可以看作增强网络能力的另一条路径.
- **为什么学**: 现代深度学习模型经常需要从大量输入中选出与当前任务最相关的部分, 仅靠固定结构的全连接、卷积或循环连接往往不够灵活.

- **核心内容:** 注意力机制通过输入驱动或任务驱动的方式分配计算资源, 让模型在不同样本、不同时间步上关注不同信息.
- **最小例子:** 在故事问答任务中, 问题为 **Where is the football?** 时, 模型应主要关注与 football 最近一次位置变化有关的句子, 而不是平均处理整段故事.
- **最该记住:** 注意力的核心是“动态选择”: 当前查询不同, 被关注的输入位置和权重也应随之改变.

1. 从网络能力到信息选择

- 前面已经学习过全连接前馈网络、卷积网络、循环网络和图神经网络等结构. 这些结构增强模型能力的方式各不相同: 全连接层增强全局混合能力, 卷积层利用局部连接和参数共享, 循环网络按时间递推建模序列, 图网络则沿图结构传播信息.
- 注意力机制提供了另一种思路: 不固定规定每个位置应该和哪些位置交互, 而是让模型根据当前输入和任务目标动态决定应该关注什么. 因此, 注意力不仅是一个网络模块, 也是一种信息筛选和资源分配机制.
- 以 Facebook bAbI tasks 中的故事问答为例, 若给出若干句人物移动记录, 再问 football 在哪里, 模型需要从多条事件中找出与 football 最后一次移动相关的句子. 若所有句子被同等处理, 关键信息很容易被无关事件淹没.

2. 人类注意力带来的启发

- 人脑每个时刻接收大量多模态信息, 包括视觉、听觉和触觉等输入. 单就视觉而言, 视网膜会持续向视觉系统发送大量信号, 但人的意识并不会同时精细处理所有信息.
- 注意力可以缓解信息超载. 例如, 在嘈杂的鸡尾酒会上, 人通常可以聚焦朋友的谈话并忽略背景噪声; 但当背景声音中出现自己的名字时, 注意力又可能迅速转移. 这说明注意力既受当前目标影响, 也会受到显著刺激触发.
- 这种现象并不意味着神经网络完全模拟了人脑机制. 在深度学习中, 注意力通常被实现为一个可微的权重计算过程, 它借鉴“选择性处理”的思想, 但主要目标仍是提高任务性能和表示能力.

3. 自下而上与自上而下

- **自下而上 (bottom-up)** 的注意力由输入数据驱动. 图像中的高对比度区域、文本中的异常词语或序列中的突变信号, 都可能因为自身显著性而获得更高关注.
- **自上而下 (top-down)** 的注意力由任务目标、问题或先验知识驱动. 当问题问“黑色牌的数字之和是多少”时, 关注对象应由“黑色牌”这一目标决定, 而不是由画面中最醒目的红色牌决定.
- 课堂上可以把二者与两种汇聚方式联系起来理解: pooling 更像从局部区域中汇聚统计信息, focus 则更强调围绕当前目标主动聚焦. 在神经网络中, 注意力往往同时包含这两种因素: 输入特征提供候选信息, 查询向量提供当前目标.

4. 硬性注意力与软性注意力的直觉

- 硬性注意力 (hard attention) 直接选择某个位置或少数位置, 类似“只看这里”. 它的选择结果通常是离散的, 因而不容易直接用普通反向传播训练.

- 软性注意力 (soft attention) 为所有候选位置分配连续权重, 类似“每个位置都看一点, 但重要位置权重重大”. 因为加权求和是可微的, 软性注意力可以方便地嵌入端到端神经网络.
- 需要注意, 软性注意力并不等于没有选择. 当权重分布很尖锐时, 它也可以近似地表现为对少数位置的聚焦; 当权重较均匀时, 它则更像一种上下文平均.

问题思考:

- 在故事问答、图像描述和机器翻译中, “当前需要关注什么”分别由什么信息决定?
- 为什么硬性注意力更接近直觉上的选择, 但软性注意力更容易训练?
- 注意力权重能否直接等同于人类意义上的解释? 为什么还需要谨慎?

3 人工神经网络的注意力机制

本节导读:

- **本节目的:** 用统一公式说明神经网络中的注意力如何计算, 并区分常见变体.
- **为什么学:** 后续自注意力、Transformer 和大语言模型都建立在注意力分布、QKV 表示和多头并行这几个基本部件上.
- **核心内容:** 注意力通常先计算查询与候选输入之间的相关性得分, 再通过 softmax 得到权重, 最后对值向量加权求和.
- **最小例子:** 给定一个查询向量 q 和三个候选记忆 $(k_1, v_1), (k_2, v_2), (k_3, v_3)$, 若 q 与 k_2 最相似, 输出就应更接近 v_2 .
- **最该记住:** Query 决定“我要找什么”, Key 决定“我能被怎样匹配”, Value 决定“匹配后取走什么信息”.

1. 两步计算: 打分与汇聚

- 设输入信息为 $\mathbf{x}_1, \dots, \mathbf{x}_N$, 查询向量为 $\mathbf{q}$. 注意力机制的第一步是计算每个输入与查询之间的相关性得分:

$$e_i = s(\mathbf{x}_i, \mathbf{q}), \quad i = 1, \dots, N.$$

其中 $s(\cdot, \cdot)$ 称为打分函数 (score function), 它可以由点积、双线性形式或一个小神经网络实现.

- 第二步是把得分归一化为注意力分布:

$$\alpha_i = \frac{\exp(e_i)}{\sum_{j=1}^N \exp(e_j)}, \quad \sum_{i=1}^N \alpha_i = 1.$$

α_i 越大, 表示第 i 个输入对当前查询越重要.

- 最后对输入表示或值向量进行加权求和:

$$\mathbf{c} = \sum_{i=1}^N \alpha_i \mathbf{x}_i.$$

这个输出 $\mathbf{c}$ 通常称为上下文向量 (context vector), 它把与当前查询相关的信息压缩到一个向量中.

2. 常见注意力打分函数

- **加性模型 (additive attention)** 使用一个小前馈网络计算相关性:

$$s(\mathbf{x}_i, \mathbf{q}) = \mathbf{v}^\top \tanh(\mathbf{W}_x \mathbf{x}_i + \mathbf{W}_q \mathbf{q}).$$

它的表达能力较强, 但计算开销相对更大.

- **点积模型 (dot-product attention)** 直接计算两个向量的内积:

$$s(\mathbf{x}_i, \mathbf{q}) = \mathbf{x}_i^\top \mathbf{q}.$$

当向量已经处在同一表示空间时, 点积可以高效衡量相似性.

- **缩放点积模型 (scaled dot-product attention)** 在点积后除以 $\sqrt{d_k}$:

$$s(\mathbf{x}_i, \mathbf{q}) = \frac{\mathbf{x}_i^\top \mathbf{q}}{\sqrt{d_k}}.$$

这里 d_k 是键向量维度. 缩放的作用不是神秘技巧, 而是防止高维点积数值过大导致 softmax 过早饱和, 从而使梯度变小、训练变慢.

- **双线性模型 (bilinear attention)** 引入可学习矩阵:

$$s(\mathbf{x}_i, \mathbf{q}) = \mathbf{x}_i^\top \mathbf{W} \mathbf{q}.$$

它比普通点积多了一层可学习的相似性度量, 但参数量也更大.

3. 键值对注意力 (Key-Value Pair Attention)

- 在更一般的注意力模型中, 每个输入位置不只提供一个向量, 而是提供键和值两类表示. 键 $\mathbf{k}_i$ 用来与查询 $\mathbf{q}$ 匹配, 值 $\mathbf{v}_i$ 用来参与最终汇聚.
- 设键矩阵和值矩阵分别为

$$\mathbf{K} = \begin{bmatrix} \mathbf{k}_1^\top \\ \vdots \\ \mathbf{k}_N^\top \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} \mathbf{v}_1^\top \\ \vdots \\ \mathbf{v}_N^\top \end{bmatrix}.$$

对单个查询, 缩放点积注意力可写为

$$\text{Attention}(\mathbf{q}, \mathbf{K}, \mathbf{V}) = \sum_{i=1}^N \alpha_i \mathbf{v}_i, \quad \alpha_i = \frac{\exp(\mathbf{q}^\top \mathbf{k}_i / \sqrt{d_k})}{\sum_{j=1}^N \exp(\mathbf{q}^\top \mathbf{k}_j / \sqrt{d_k})}.$$

- 这里容易混淆的是: Key 和 Value 并不一定是两个不同的原始输入. 在许多模型中, 它们由同一个输入向量经过不同线性变换得到. 这样做的目的, 是让“用于匹配的信息”和“最终被取出的信息”可以分别学习.

4. 多头注意力 (Multi-Head Attention)

- 单个注意力头只能在一个表示子空间中计算相关性。多头注意力使用多组查询、键和值投影并行计算注意力, 让不同头关注不同关系。
- 对矩阵形式的查询 Q 、键 K 和值 V , 第 m 个头可写为

$$\text{head}_m = \text{Attention}(QW_m^Q, KW_m^K, VW_m^V).$$

多头输出再拼接并线性变换:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O.$$

- 从教学直觉看, 一个头可能关注语法依赖, 另一个头可能关注指代关系, 还有的头可能关注局部邻近词。这种分工不是人工指定的, 而是在训练中由任务损失推动形成。

5. 结构化注意力与指针网络

- 结构化注意力 (structural attention) 把注意力分布限制在某种结构上, 例如树、图、片段或路径。它适合处理输入内部存在明确依赖关系的任务, 如句法结构、图结构关系或多步推理。
- 指针网络 (pointer network) 可以只使用注意力的第一步, 即把注意力分布本身作为输出。此时 α_i 不再只是中间权重, 而是表示“答案指向输入第 i 个位置”的概率。
- 指针网络适合输出来自输入元素的任务, 例如排序、组合优化、抽取式问答中的答案边界选择等。这一点与普通分类器不同: 普通分类器在固定类别集合中选择标签, 指针网络则在当前输入的位置集合中选择答案。

问题思考:

- 为什么 Q 、 K 、 V 三者要分开学习, 而不是直接用同一个向量完成所有事情?
- 缩放点积注意力中的 $\sqrt{d_k}$ 如果去掉, 在高维情况下可能带来什么训练问题?
- 指针网络的输出空间为什么会随输入长度变化?

4 注意力机制的应用

本节导读:

- **本节目的:** 通过典型任务说明注意力机制如何把“相关信息选择”落到实际模型中。
- **为什么学:** 注意力并不只属于 Transformer, 它曾广泛用于 RNN 编码器-解码器、图像描述、阅读理解和文档分类等模型。
- **核心内容:** 不同任务中的查询、键和值含义不同, 但计算逻辑都是先找相关位置, 再汇聚相关信息。
- **最小例子:** 做情感分类时, `The infantile cart is easy to use.` 这句话中, 模型可能需要同时关注 `infantile` 的负面语义和 `easy to use` 的正面语义, 再综合判断整体情感。
- **最该记住:** 应用注意力时, 关键不是套公式, 而是明确谁在查询、候选信息是什么、最终要汇聚成什么表示。

1. 文本分类

- 在文本分类中, 输入是一串词或 token, 输出是类别标签, 如情感正负、主题类别或垃圾邮件判别. 注意力可以给每个词分配权重, 得到句子表示:

$$\mathbf{r} = \sum_{t=1}^T \alpha_t \mathbf{h}_t.$$

其中 $\mathbf{h}_t$ 是第 t 个词的上下文表示, α_t 表示该词对分类任务的重要程度.

- 与简单平均所有词向量相比, 注意力更能突出关键词或关键短语. 例如在电影评论中, “not good” 与 “good” 的作用不同, 模型不能只看到单个积极词, 还要结合上下文.
- 需要注意, 注意力热力图可以帮助观察模型关注了哪些位置, 但它不必然等同于严格因果解释. 真正判断某个词是否决定预测, 还需要遮挡、反事实输入或梯度等分析手段.

2. 层次注意力模型

- 对长文档而言, 直接对所有词做一个注意力分布可能过于粗糙. 层次注意力模型通常先在句子内部做词级注意力, 得到每个句子的表示, 再在文档层面对句子表示做句级注意力.
- 若第 i 个句子包含词表示 $\mathbf{h}_{it}$, 词级注意力可写为

$$\mathbf{s}_i = \sum_t \alpha_{it} \mathbf{h}_{it}.$$

再对句子表示做注意力:

$$\mathbf{d} = \sum_i \beta_i \mathbf{s}_i.$$

- 这种结构符合文档的自然层次: 词组成句子, 句子组成文章. 对新闻分类、评论分析和长文本检索等任务, 它能减少长序列建模中的信息稀释问题.

3. 机器翻译

- 在传统编码器-解码器 RNN 中, 编码器常把整个源句压缩成一个固定长度向量, 解码器再根据这个向量生成目标句. 当句子较长时, 单个向量容易成为信息瓶颈.
- 引入注意力后, 解码器在生成第 t 个目标词时, 用当前解码状态 $\mathbf{s}_t$ 作为查询, 对源句所有编码状态 $\mathbf{h}_1, \dots, \mathbf{h}_N$ 计算权重:

$$\mathbf{c}_t = \sum_{i=1}^N \alpha_{ti} \mathbf{h}_i.$$

这样每个目标词都有自己的上下文向量 $\mathbf{c}_t$.

- 从翻译角度看, 注意力矩阵可以近似反映源语言和目标语言之间的词对齐关系. 传统统计机器翻译也依赖对齐思想, 神经注意力机制则把对齐和翻译放进同一个端到端模型中学习.

4. 图像描述与视觉注意力

- 在图像描述任务中, 模型需要根据图像生成自然语言描述. 卷积网络或视觉编码器先提取多个区域特征, 解码器在生成每个词时对这些区域做注意力.

- 例如生成单词“bird”时，注意力应更多集中在鸟所在区域；生成“water”时，权重又可能转移到水面区域。这说明注意力可以把视觉区域与语言生成过程联系起来。
- 视觉注意力还常用于目标检测、视觉问答和多模态模型。在这些任务中，查询可能来自文本问题，候选信息可能来自图像区域，因而注意力成为跨模态信息交互的桥梁。

5. 阅读理解与双向注意力

- 在阅读理解任务中，输入通常包含一段文章和一个问题。模型既要让问题关注文章中的相关片段，也要让文章表示吸收问题信息。
- 以 BiDAF 这类模型为例，它同时建模 context-to-query attention 和 query-to-context attention。前者回答“文章中每个位置应该看问题的哪些词”，后者回答“问题整体最相关的文章位置在哪里”。
- 对抽取式问答而言，注意力分布还可以进一步用于预测答案起点和终点。这与指针网络思想相通：模型不是从固定类别中生成答案，而是在文章位置中选择答案边界。

问题思考：

- 在文本分类、机器翻译和图像描述中，注意力的查询分别来自哪里？
- 为什么机器翻译中的注意力矩阵可以被看作一种软对齐？
- 在阅读理解中，只让问题关注文章是否足够？为什么还常需要文章和问题的双向交互？

5 自注意力模型

本节导读：

- **本节目的：**说明自注意力如何在同一序列内部建立全局依赖，并给出矩阵化计算方式。
- **为什么学：**Transformer 的核心就是多头自注意力。不理解自注意力，就很难理解后续语言模型如何利用上下文。
- **核心内容：**对同一输入序列分别生成 Q 、 K 、 V ，再用 QK^T 得到任意两位置之间的动态连接权重。
- **最小例子：**在“The weather is nice today”中，处理“nice”时，模型可以直接关注“weather”和“today”，不必像 RNN 那样一步步传递信息。
- **最该记住：**自注意力把序列中的每个位置都看作一次查询，让每个位置都能从所有位置汇聚上上下文。

1. 从局部依赖到非局部依赖

- 卷积网络擅长提取局部模式。一层卷积只连接邻近位置，多层叠加后感受野逐渐扩大。这种设计计算高效，但远距离信息需要经过多层才能交互。
- 循环网络按时间顺序处理序列，理论上可以把前面信息传到后面，但远距离依赖需要经过很多递推步骤，训练中容易受到梯度衰减、状态容量和并行效率的限制。
- 全连接层可以让所有位置直接相连，但它通常要求固定输入长度，参数量也随长度变化而膨胀。更重要的是，全连接权重是固定参数，不能根据当前样本动态改变连接强度。

- 自注意力解决的正是非局部依赖问题. 它让任意两个位置之间都可以通过注意力权重 α_{ij} 直接交互, 且这个权重由当前输入动态生成.

2. 自注意力的矩阵表示

- 设输入序列长度为 L , 每个位置的表示维度为 d_{model} . 将输入写成矩阵

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_L^\top \end{bmatrix} \in \mathbb{R}^{L \times d_{\text{model}}}.$$

通过三组可学习线性变换得到

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^V.$$

其中 $\mathbf{Q}, \mathbf{K} \in \mathbb{R}^{L \times d_k}$, $\mathbf{V} \in \mathbb{R}^{L \times d_v}$.

- 缩放点积自注意力为

$$\mathbf{H} = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \right) \mathbf{V}.$$

这里 softmax 对每一行进行归一化, 因而第 i 行表示第 i 个位置作为查询时对所有位置的注意力分布.

- 展开到单个位置, 有

$$\mathbf{h}_i = \sum_{j=1}^L \alpha_{ij} \mathbf{v}_j, \quad \alpha_{ij} = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d_k})}{\sum_{\ell=1}^L \exp(\mathbf{q}_i^\top \mathbf{k}_\ell / \sqrt{d_k})}.$$

这说明每个输出位置都是全部值向量的加权和, 权重由该位置的查询和其他位置的键决定.

3. 自注意力与普通注意力的关系

- 普通键值对注意力中, 查询可以来自解码器状态, 键和值可以来自编码器输出. 自注意力的特殊之处在于, 查询、键和值都由同一个输入序列生成.
- 这并不表示 $\mathbf{Q}$ 、 $\mathbf{K}$ 、 $\mathbf{V}$ 完全相同. 它们来自同一个 $\mathbf{X}$, 但经过不同参数矩阵投影, 因而分别承担“提问”“匹配”和“提供内容”的角色.
- 自注意力天然适合变长序列: 参数矩阵 $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V$ 不依赖序列长度, 长度变化只会改变注意力矩阵的大小.

4. 多头自注意力

- 多头自注意力将输入投影到多个子空间中分别做自注意力. 如果总维度固定, 每个头通常使用较小的 d_k 和 d_v , 最后再拼接回模型维度.
- 多头机制的价值在于同时捕获多种关系. 例如, 在语言序列中, 有的头可能偏向相邻词搭配, 有的头可能偏向主谓依赖, 有的头可能偏向指代或长距离语义关联.
- 多头数量并不是越多越好. 头数增加会改变每个头的维度和计算模式, 实际效果还取决于模型规模、数据量、任务以及训练稳定性.

5. 位置信息与掩码

- 自注意力本身只比较内容相似性. 如果不加入位置信息, 同一组 token 的排列顺序改变后, 模型很难区分“我爱你”和“你爱我”这样的差异.
- 因此, Transformer 通常会在输入表示中加入位置编码 (positional encoding) 或学习得到的位置嵌入. 位置信息告诉模型每个 token 处在序列中的哪个位置.
- 在解码式语言模型中, 还需要因果掩码 (causal mask), 防止当前位置在预测下一个 token 时看到未来 token. 对 padding 位置也常使用 padding mask, 避免模型把补齐符号当作真实内容.

问题思考:

- 为什么自注意力能直接建模长距离依赖, 但计算和显存开销会随序列长度平方增长?
- 如果去掉位置编码, 自注意力模型在处理自然语言时会丢失哪些关键信息?
- 多头注意力学习到的不同头, 是否一定都有清晰的人类可解释含义?

6 Transformer 简介

本节导读:

- **本节目的:** 从自注意力出发理解 Transformer 块的完整结构, 并比较它与 CNN、RNN 的建模差异.
- **为什么学:** Transformer 是当前自然语言处理和大语言模型的基础结构, 其成功来自注意力、并行计算和深层残差网络的组合.
- **核心内容:** 一个 Transformer 块通常由多头注意力、逐位前馈网络、残差连接和层归一化组成, 并依赖位置编码保留顺序信息.
- **最小例子:** 编码器处理一句话时, 每一层先让所有 token 通过自注意力交换信息, 再用同一个前馈网络分别更新每个位置的表示.
- **最该记住:** Transformer 并不是“只有注意力”, 而是“注意力 + 位置 + 残差 + 归一化 + 前馈网络”的稳定深层结构.

1. Transformer Encoder 的基本组成

- 仅靠自注意力还不够. 自注意力负责跨位置的信息交互, 但模型还需要知道顺序、稳定训练、增强非线性表达, 并让深层网络中的梯度顺利传播.
- **位置编码:** 向 token 表示中注入顺序信息. 它可以是固定的正弦余弦编码, 也可以是可学习的位置嵌入.
- **多头自注意力:** 让每个位置从整句或整段上下文中汇聚相关信息.
- **残差连接:** 将子层输入直接加到子层输出上, 形式上可写为 $\mathbf{x} + \text{Sublayer}(\mathbf{x})$. 直连边可以缓解深层网络中信息和梯度难以传播的问题.

- **层归一化 (Layer Normalization)**: 对单个样本内部的特征维度做归一化, 提升训练稳定性. 在现代实现中, LayerNorm 可以放在子层前 (pre-norm) 或子层后 (post-norm), 不同选择会影响深层模型训练.
- **逐位前馈网络 (position-wise FFN)**: 对每个位置独立应用同一组前馈网络, 常见形式为

$$\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2.$$

它不负责位置间通信, 而负责在每个位置上做非线性特征变换.

2. 复杂度与路径长度对比

- 设序列长度为 L , 表示维度为 d , 卷积核大小为 k . 在常见简化分析下, CNN、RNN 和 Transformer 的对比如下:

模型	每层复杂度	序列操作数	最大路径长度
CNN	$O(kLd^2)$	$O(1)$	$O(\log_k L)$
RNN	$O(Ld^2)$	$O(L)$	$O(L)$
Transformer	$O(L^2d)$	$O(1)$	$O(1)$

- 序列操作数衡量模型能否并行处理序列. RNN 必须按时间步递推, 因而并行性较弱; CNN 和 Transformer 每层都可以并行计算所有位置.
- 最大路径长度衡量两个远距离位置之间需要经过多少计算步骤才能交互. Transformer 中任意两个位置在同一层自注意力中即可直接交互, 因此路径长度为 $O(1)$.
- Transformer 的主要代价是注意力矩阵大小为 $L \times L$, 因而长序列场景下计算和显存开销明显. 这也是稀疏注意力、局部注意力、线性注意力和缓存机制等方法被提出的重要原因.

3. 编码器-解码器、仅编码器与仅解码器

- 原始 Transformer 用于机器翻译, 包含编码器和解码器. 编码器读取源语言句子, 解码器在已生成目标词的基础上生成下一个目标词, 并通过交叉注意力关注编码器输出.
- 仅编码器模型适理解类任务. BERT 属于这一类, 它使用双向自注意力读取整句上下文, 常通过掩码语言模型 (Masked Language Model, MLM) 等预训练任务学习表示.
- 仅解码器模型适合生成类任务. GPT 类模型使用因果自注意力, 每次只能看到当前位置之前的上下文, 通过预测下一个 token 学习语言分布.
- 编码器-解码器模型适合输入和输出都较复杂的条件生成任务, 如机器翻译、摘要和多模态生成. 仅编码器、仅解码器和编码器-解码器并无绝对优劣, 关键取决于任务目标.

4. 从词袋模型到 Transformer

- 词袋模型 (Bag-of-Words) 把文本表示为词频或计数向量. 它简单有效, 但忽略词序, 也难以表达上下文中的语义变化.
- Word2vec 等静态词向量方法把词映射到稠密向量空间, 可以捕捉一定语义关系. 但同一个词通常只有一个向量, 无法区分 “bank” 在 “银行” 和 “河岸” 中的不同含义.
- RNN 将文本作为序列处理, 通过隐藏状态逐步整合上下文. 它能表达顺序, 但长距离依赖和并行训练仍受限制.

- Transformer 通过自注意力为每个 token 构造上下文表示. 同一个词在不同句子中会根据周围上下文得到不同表示, 因而更适合复杂语言理解和生成.

5. BERT 与 GPT 的代表性差异

- BERT (Bidirectional Encoder Representations from Transformers) 是仅编码器模型的典型代表. 它利用双向上下文形成表示, 常用于文本分类、命名实体识别、问答匹配等理解任务.
- GPT (Generative Pre-trained Transformer) 是仅解码器模型的典型代表. 它采用自回归方式建模:

$$p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t | x_{<t}).$$

训练时目标是根据前文预测下一个 token, 推理时则不断把已生成 token 追加到上下文中.

- 二者最核心的区别在于可见上下文. BERT 训练表示时可同时利用左右上下文, GPT 生成第 t 个 token 时只能使用 t 之前的信息. 因此, BERT 更偏理解表示, GPT 更偏连续生成.

问题思考:

- Transformer 为什么比 RNN 更容易并行训练?
- 为什么 Transformer 仍然需要位置编码?
- BERT 的双向上下文和 GPT 的因果上下文分别适合什么任务?

7 生成式模型的细节

本节导读:

- **本节目的:** 以生成式语言模型为例, 拆解 Tokenizer、Transformer Blocks 和 LM Head 三个核心部件.
- **为什么学:** 大语言模型常被整体称为一个模型, 但实际推理过程包含分词、嵌入、上下文更新、概率预测和采样等多个步骤.
- **核心内容:** 文本先被切成 token 并映射为向量, 经过多层 Transformer 块得到上下文表示, 最后由语言建模头预测下一个 token 的概率分布.
- **最小例子:** 输入 I love llamas 后, 模型把它切成若干 token, 计算上下文表示, 再预测下一个 token 可能是 because, 标点符号 ., 或其他词片段.
- **最该记住:** 生成式语言模型不是一次生成完整答案, 而是反复执行“读上下文, 预测下一个 token, 把 token 接回上下文”的过程.

1. 整体架构: Tokenizer, Transformer Blocks, LM Head

- PPT 中把生成式 Transformer LLM 概括为三个主要组成部分: Tokenizer、Transformer Blocks 和 LM Head.

- Tokenizer 负责把原始字符串转成 token ID, Transformer Blocks 负责把 token 序列转成上下文化隐藏表示. LM Head 负责把最后一层隐藏表示投影到词表大小的 logits, 再经 softmax 得到下一个 token 的概率分布.
- 对自回归语言模型, 训练目标通常是最大化真实下一个 token 的条件概率, 或等价地最小化交叉熵损失:

$$\mathcal{L} = - \sum_{t=1}^T \log p(x_t | x_{<t}).$$

2. 分词与嵌入 (Tokenization & Embedding)

- Tokenizer 的作用不是简单按空格切词. 现代语言模型常使用 BPE、WordPiece 或 SentencePiece 等子词方法, 把常见词作为整体, 把罕见词拆成更小片段.
- 词表 (Token Vocabulary) 给每个 token 分配一个整数 ID, 输入文本被编码成 token ID 序列后, 模型通过嵌入矩阵查表得到 token 向量:

$$\mathbf{e}_t = \mathbf{E}[x_t].$$

其中 $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$, $|\mathcal{V}|$ 为词表大小.

- 这里容易混淆的是 token 和词并不等价. 一个英文单词可能是一个 token, 也可能被拆成多个 token; 中文中一个字、一个词或一个子词片段都可能成为 token. 因而模型的上下文长度本质上是 token 长度, 不是自然语言中的词数.

3. Transformer 块中的前馈网络直觉

- 自注意力负责从上下文中拿信息, 逐位前馈网络负责对每个位置已经汇聚好的信息做非线性变换. 它可以理解为对每个 token 表示独立应用的多层感知器.
- 典型 FFN 先把维度从 d_{model} 扩展到更高维, 经过激活函数, 再投影回原维度. 这种“升维-非线性-降维”结构增强了每个位置的表达能力.
- 在许多 Transformer 变体中, FFN 的激活函数会使用 GELU、SwiGLU 等形式. 这些选择与前面学习的激活函数知识相连: 它们既影响表示能力, 也影响优化稳定性.

4. 自注意力如何利用上下文

- 生成式模型在处理当前位置时, 会把当前 token 表示作为查询, 与此前所有 token 的键进行相关性计算, 再对值向量加权求和. 因果掩码保证它不能看到未来 token.
- 例如当模型已读到 “I love llamas because” 时, 生成下一个 token 需要综合 “I”、“love”、“llamas” 和 “because” 等上下文信息. 注意力使当前位置可以直接从相关历史位置取信息.
- 在实际推理中, 为了避免每生成一个 token 都重复计算全部历史位置, 模型会缓存 (cache) 历史 token 的 Key 和 Value. 新 token 只需计算自己的 Query、Key、Value, 再与缓存的 Key-Value 做注意力, 这可以显著加速自回归生成.

5. 语言建模头与解码策略

- LM Head 将隐藏状态 $\mathbf{h}_t$ 映射为词表上每个 token 的得分:

$$\mathbf{z}_t = \mathbf{W}_{\text{lm}} \mathbf{h}_t + \mathbf{b}_{\text{lm}}, \quad p(x_{t+1} | x_{\leq t}) = \text{softmax}(\mathbf{z}_t).$$

- 贪婪解码 (greedy decoding) 每一步选择概率最大的 token, 生成结果稳定, 但容易陷入重复或过于保守.
- 温度采样 (temperature sampling) 使用

$$p_i = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}.$$

当温度 τ 较低时, 分布更尖锐, 输出更确定; 当 τ 较高时, 分布更平坦, 输出更多样.

- Top- k 采样只在概率最高的 k 个候选中采样, Top- p 或 nucleus sampling 则选择累计概率达到 p 的最小候选集合. 这些方法都在“质量”和“多样性”之间做折中.

6. 上下文长度、参数规模与稀疏注意力

- 上下文长度 (context length) 决定模型一次能读取多少 token. 更长上下文能容纳更多历史信息, 但普通自注意力的 $O(L^2)$ 开销会随长度迅速增加.
- 参数规模决定模型可学习的函数复杂度, 但更大参数量并不自动保证更好效果. 数据质量、训练目标、优化方法、推理策略和对齐方式都会影响模型行为.
- 为了处理长序列, 研究中常使用局部注意力、块稀疏注意力、滑动窗口注意力或混合全局 token 等策略. 其共同目标是在保留重要上下文交互的同时降低注意力矩阵的计算成本.

7. 参考论文与模型例子

- “Attention Is All You Need” 提出了标准 Transformer 架构, 是理解本讲内容的基础论文.
- BERT 展示了仅编码器 Transformer 在语言理解预训练中的作用; GPT 系列展示了仅解码器 Transformer 在自回归生成中的能力.
- PPT 中提到的 Llama 3 等模型论文可以作为现代大规模生成式模型的结构和训练案例阅读. 阅读这类论文时, 应重点关注模型结构、训练数据、上下文长度、tokenizer、推理策略和评测方式, 而不只是参数规模.

问题思考:

- 为什么语言模型需要 tokenizer, 而不能直接把原始字符串交给 Transformer?
- 生成式模型中因果掩码的作用是什么? 如果去掉它会发生什么?
- 温度、Top- k 和 Top- p 如何影响生成结果的稳定性与多样性?

8 小结

- 注意力机制的核心是根据查询与候选信息之间的相关性动态分配权重, 再对值向量进行加权汇聚.
- 自注意力把这种机制用于同一序列内部, 使每个位置都可以直接利用其他位置的信息, 是 Transformer 的核心.
- Transformer 的成功不仅来自自注意力, 还依赖位置编码、残差连接、层归一化、逐位前馈网络和高效并行训练.

- 生成式语言模型把文本分成 token, 经过多层 Transformer 块得到上下文表示, 再通过语言建模头逐步预测下一个 token.

(注: 详细定义和更多内容请参考课程教材第 8 章)